

RDD: Real-time Dynamic Detection of APT Attacks Based on Provenance Graph Matching

1st Yuzhang Tian

School of Computer Science and Technology
Taiyuan University of Technology
 Taiyuan, China
 yu36113@gmail.com

2nd Guohua Xin

School of Cyber Security
Tianjin University
 Tianjin, China
 0000-0002-9525-0728

3rd Cong Wang

School of Artificial Intelligence
Tianjin University of Science and Technology
 Tianjin, China
 wangcongjcdd@tust.edu.cn

4th Tuoyu Chen

School of Cyber Security
Tianjin University
 Tianjin, China
 chenty_9920@tju.edu.cn

5th Yuebin Wei

School of Artificial Intelligence
Tianjin University of Science and Technology
 Tianjin, China
 kanekilover1258@gmail.com

Jianhua Wang*

School of Computer Science and Technology
Taiyuan University of Technology
 Taiyuan, China
 wangjianhua02@tyut.edu.cn
 0000-0002-2773-3429

Abstract—Advanced Persistent Threats (APTs) are stealthy and long-term cyberattacks designed to exfiltrate sensitive data or disrupt critical systems through multi-stage infiltration. Existing detection methods often rely on provenance-graph matching, where system entities and their interactions are represented as nodes and edges to capture data provenance. However, these methods face challenges due to the large scale of provenance graphs, the computational complexity of graph matching, and the lack of real-time, dynamic detection to support streaming analysis and temporal evidence accumulation. We present RDD, a real-time system that adopts streaming input for host monitoring and introduces a novel matching algorithm based on provenance graphs. By combining log-likelihood ratio computation with Bayesian updating, RDD treats each matching result as quantifiable evidence to refine the posterior probability of ongoing anomalies. Evaluation on three public datasets shows that RDD delivers the highest accuracy of 95.69%, substantially outperforming POIROT (79.66%) and THREATTRACE (91.56%).

Index Terms—Provenance Graph Matching, APT Detection, Real-time Detection, Bayesian Updating, Log-likelihood Ratio.

I. INTRODUCTION

Against the backdrop of rapid development in network technologies, Advanced Persistent Threats (APTs) have become a significant form of cyber threat, characterized by high stealth and potential for severe damage. During their latent phase, attackers continuously exfiltrate sensitive information, monitor system operations, and may even manipulate critical resources. The attack chains are complex and difficult to detect. To address the challenge of threat detection, many studies focus on provenance-graph-based approaches [1]–[4], where kernel audit logs are constructed into provenance graphs [5]. Malicious behaviors, such as information flows and causal dependencies, are extracted to build query graphs, which serve as robust indicators for search [6], thereby modeling threat

detection as a graph matching problem—i.e., searching for subgraphs within a large provenance graph that resemble the connections specified in the query graph. For example, the POIROT system [7] proposes a graph matching algorithm capable of resisting evasive attacks while achieving effective alignment results.

However, despite their usefulness, these methods have the following limitations:

- The provenance graph is typically large in scale, and graph matching algorithms are computationally complex with numerous possible subgraph combinations, making the graph matching process highly challenging.
- Existing studies generally lack attention to real-time and dynamic detection, making it difficult to support streaming analysis and temporal evidence accumulation.

We design RDD to address these challenges. Unlike traditional methods that rely on exhaustive expansion and traversal of the provenance graph to complete the query graph and identify aligned subgraphs, we propose a node alignment scoring mechanism to measure the alignment quality between a node in the provenance graph and a node in the query graph. By selecting, for each query graph node, the corresponding node with the highest score and combining these optimal alignments, the overall optimal aligned subgraph can be efficiently obtained, achieving the goal of graph matching.

RDD is a real-time system deployable on long-running hosts, processing streaming inputs for on-the-fly detection. It converts graph alignment scores from different time periods into log-likelihood ratios, accumulating each as quantifiable evidence while preserving cross-time correlations. The total log-likelihood ratio is then used in a Bayesian update to dynamically estimate the posterior probability of the host being abnormal, effectively framing anomaly detection as a probabilistic estimation of whether the host is currently

* Corresponding author: Jianhua Wang

abnormal.

Our paper makes contributions summarized as follows:

- **Novel graph matching algorithm.** By replacing the traditional expansion search with a Node Alignment Score, the optimal alignment node can be directly selected from the candidate node sets, significantly reducing computational overhead.
- **Real-time dynamic detection.** By converting graph alignment scores across different time periods into log-likelihood ratios and accumulating them, and then applying Bayesian updates to dynamically estimate the posterior probability of host abnormality, the system achieves online anomaly detection under streaming inputs.

This paper is organized as follows. Related work is presented in §II. The system framework is introduced in §III. Evaluation and experimental results are discussed in §IV. Finally, we conclude the paper in §V.

II. RELATED WORK

Most existing research on APT attack detection mainly focuses on improving detection accuracy [8], [9], with relatively little attention paid to developing truly dynamic real-time detection systems. In terms of dynamic monitoring, some studies have adopted streaming input approaches. For instance, trace is a node-level real-time detection system that employs a streaming input mode. Instead of raising alerts immediately upon detecting anomalies, it continuously updates the status of anomalous nodes within a predefined time threshold and only triggers alerts when specific conditions are met, thereby achieving dynamic detection. However, such methods are limited in their ability to effectively correlate anomalous evidence detected at different points in time, leading to insufficient evidence correlation.

A. Provenance-Based Threat Detection

In host-based threat detection, data provenance has received significant attention due to its ability to comprehensively record system events. Existing studies can be broadly categorized into two types: misuse-based methods and anomaly-based methods.

Misuse-based methods detect abnormal behaviors by leveraging known threat indicators. For example, ProvG-Searcher [10] leverages provenance graphs and employs graph representation learning to embed subgraphs into a vector space, where the relationships between subgraphs can be directly evaluated. These approaches rely on expert knowledge to model common attacker TTPs (tactics, techniques, and procedures) as query graphs, which are then matched against the provenance graph to identify threats. Such methods require predefined threat indicators and share similarities with our approach.

Anomaly-based methods, in contrast, learn models of normal behavior and identify deviations as anomalies. For instance, Unicorn [11] employs the WL-kernel method to extract global graph features and combines it with an evolutionary model to detect attack graphs. The core idea is to measure graph similarity using graph kernels and leverage contextual

analysis. However, due to the limitations of graph kernel expressiveness, these methods often struggle to detect evasive attacks.

B. Anomaly Tracing

On monitored hosts, simply generating alerts upon detecting an attack is insufficient; it is crucial to further trace the precise location of anomalous activities. Some anomaly-based systems [12] issue alerts when the system exhibits abnormal behavior but are limited in accurately pinpointing the anomaly. In contrast, detection systems capable of anomaly tracing are often misuse-based, such as Holmes [13], which relies on prior knowledge of anomalous graph patterns. This aspect bears similarity to our proposed approach.

C. Real-time Detection

Most existing research on APT attack detection mainly focuses on improving detection accuracy, with relatively little attention paid to developing truly dynamic real-time detection systems. In terms of dynamic monitoring, some studies have adopted streaming input approaches. For instance, THREATTRACE [14] is a node-level real-time detection system that employs a streaming input mode. Instead of raising alerts immediately upon detecting anomalies, it continuously updates the status of anomalous nodes within a predefined time threshold and only triggers alerts when specific conditions are met, thereby achieving dynamic detection. However, such methods are limited in their ability to effectively correlate anomalous evidence detected at different points in time, leading to insufficient evidence correlation.

III. SYSTEM FRAMEWORK

This section describes the design of RDD. The primary components, illustrated in Fig. 1, are summarized below, with further details provided in the corresponding subsections.

- **(§III-A) Data Provenance Generator.** This module models the system audit log data to be analyzed as a provenance graph, which serves as the input for in-depth analysis by the system.
- **(§III-B) Data Storage.** This module is responsible for allocating the provenance graph data generated by the previous module to disk and memory for subsequent use. A size-limited subgraph is maintained in memory to enhance scalability.
- **(§III-C) Threat Detection.** This module takes the subgraph provided by the previous component as the target provenance graph and performs query matching with the query graph using a novel graph matching algorithm proposed in this study. The goal is to identify the optimal aligned subgraph in the provenance graph for each query graph.
- **(§III-D) Alert and Trace.** This module receives the optimal aligned subgraph and its graph alignment score from the previous component, then converts the graph alignment score into a log-likelihood ratio to serve as cumulative evidence. Whenever new alignment evidence

becomes available, the system updates the posterior probability of the host being currently abnormal.

A. Data Provenance Generator

For provenance graph generation, consistent with other provenance-graph-based threat detection methods, we use the Camflow [15] to construct a unified, system-wide provenance graph with temporal ordering.

B. Data Storage

Under the streaming input mode, newly acquired nodes and edges from the host are appended to the full graph stored on disk. Simultaneously, a subgraph is maintained in memory, consisting of active nodes, relevant nodes, and the edges among them. Here, active nodes are defined as node entities required for detection analysis, while relevant nodes are those reachable from active nodes within two hops.

C. Threat Detection

The provenance graph is streamed from the in-memory subgraph as input, and a novel provenance-graph-based matching algorithm is applied to perform graph matching between the provenance graph and the query graph. This process aims to identify the optimal aligned subgraph and compute the graph alignment score S_t . The detailed algorithmic procedure is as follows:

a) *step1*: First, for each node in the query graph, candidate nodes in the provenance graph are identified based on their entity types and stored in the corresponding candidate set.

b) *step2*: In conventional provenance-graph-based matching algorithms, this step typically requires an expansion search, i.e., performing forward or backward traversal in the provenance graph based on the candidate node set obtained in the previous step to identify the aligned subgraph. However, due to the large scale of the provenance graph, the number of possible candidate alignment combinations can become extremely large, thereby significantly increasing the computational overhead. To address this challenge, we propose a specialized graph matching algorithm in which expansion search is not performed at this step. Instead, an alignment metric referred to as the Node Alignment Score is employed to sequentially evaluate the contribution of each node in the candidate node sets to the final aligned subgraph. Consequently, the optimal node alignment can be directly selected from the candidate nodes. The computation of the Node Alignment Score is defined in (1).

$$\begin{aligned} V(i : k) &= \sum_{j:(i \rightarrow j) \in F(G_q)} (C \times I_{k \rightarrow l} + D \times \max_{m \in c(j)} (I_{k \rightarrow m})) \\ &+ \sum_{j:(j \rightarrow l) \in F(G_q)} (C \times I_{l \rightarrow k} + D \times \max_{m \in c(j)} (I_{m \rightarrow k})) \end{aligned} \quad (1)$$

where C represents $A_{\{j:l\}}$, and D represents $(1 - A_{\{j:l\}})$; $V(i : k)$ denotes the Node Alignment Score of a candidate node k in the provenance graph corresponding to a query

graph node i ; $i : k$ indicates that a query graph node i and a provenance graph node k have an alignment relationship; $j:(i \rightarrow j) \in F(G_q)$ represents all the out-neighbors node j of a node i in the query graph G_q ; $A_{\{j:l\}}$ is an indicator function that takes the value 0 or 1, specifying whether a query graph node j is fixed to a provenance graph node l ; $c(j)$ denotes the candidate node set in the provenance graph for a query graph node j ; and $I_{k \rightarrow l}$ represents the Edge Alignment Score of the path $k \rightarrow l$ in the provenance graph. This method evaluates the contribution of candidate nodes to the final graph alignment result by computing the total Edge Alignment Score of the matched outgoing and incoming edges between the query graph and the provenance graph.

The Edge Alignment Score is a path-based metric in the provenance graph, which measures the likelihood that an attacker generates a specific information flow and effectively resists evasion attempts. The computation of the Edge Alignment Score is defined in (2).

$$I_{i \rightarrow j} = \begin{cases} \max(\frac{1}{L_{i \rightarrow j}}) & L_{i \rightarrow j} \leq L_{thr} \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

where $I_{i \rightarrow j}$ denotes the Edge Alignment Score of the path $i \rightarrow j$; $L_{i \rightarrow j}$ represents the minimum number of nodes that an attacker must independently control to accomplish a specific attack step; L_{thr} denotes the maximum value of $L_{i \rightarrow j}$, which aims to ignore those paths that are considered highly difficult for an attacker to control and are generally regarded as having no abnormal risk; this computation is performed based on the number of minimum common ancestors among all nodes along the path.

c) *step3*: This step is responsible for generating the final optimal aligned subgraph and computing the Graph Alignment Score. After completing Step (2), an optimal aligned node has been selected from the candidate node set for each node in the query graph. In general, the nodes in the optimal aligned subgraph are mostly those with the highest Node Alignment Score within their candidate sets. Therefore, it can be approximately assumed that the independently selected optimal aligned nodes are connected in the provenance graph and can be combined into a subgraph. However, for rigor, a lightweight search is still required in the provenance graph during the computation of the Graph Alignment Score to verify whether the selected nodes preserve the alignment relationships corresponding to the query graph, thereby constructing the complete optimal aligned subgraph. Finally, to evaluate the alignment quality of this subgraph, the Graph Alignment Score is defined based on the Edge Alignment Score. The computation of the Graph Alignment Score is defined in (3).

$$S(G_p : G_q) = \frac{1}{F(G_q)} \sum_{(i \rightarrow j) \in F(G_q)} I_{k \rightarrow l} \quad (3)$$

where $k \rightarrow l$ is an edge in the provenance graph, which corresponds to the alignment of edge $i \rightarrow j$ in the query graph; $G_p : G_q$ denotes the aligned subgraph between the query graph G_q and the provenance graph G_p ; $F(G_q)$ represents

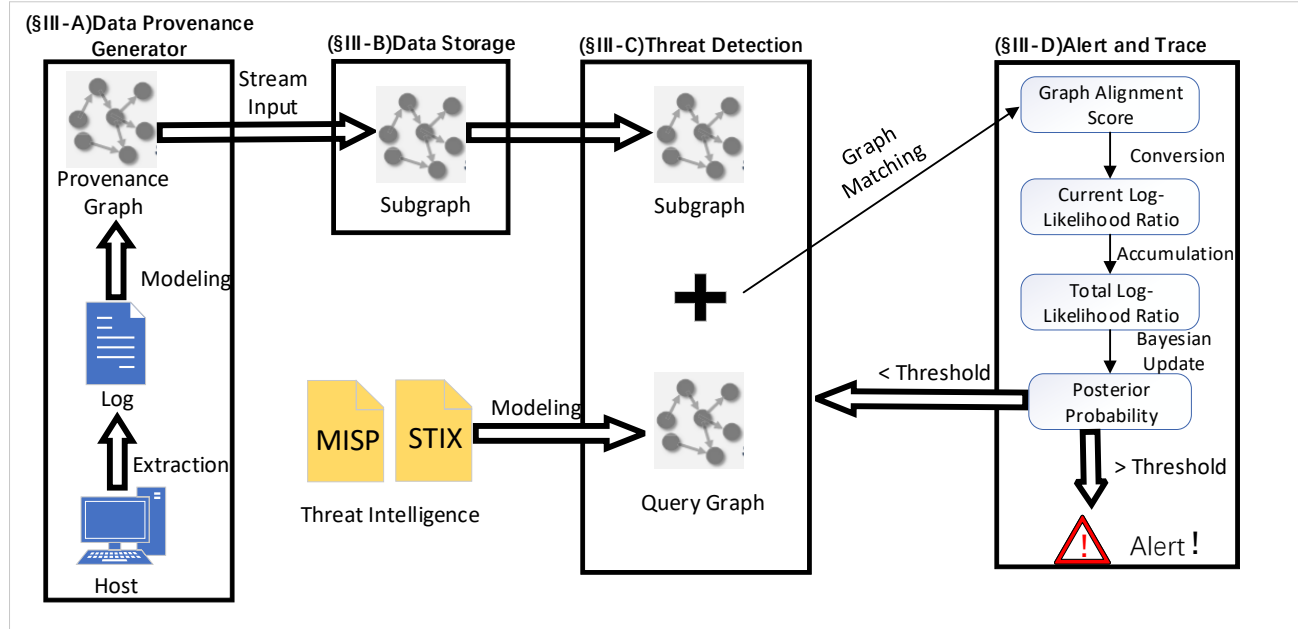


Fig. 1: Overview of components in RDD. (§III-A) Data Provenance Generator, (§III-B) Data Storage, (§III-C) Threat Detection, (§III-D) Alert and Trace.

the number of nodes in the query graph; $(i \rightarrow j) \in F(G_q)$ denotes an edge in the query graph, and $I_{k \rightarrow l}$ denotes the Edge Alignment Score of $k \rightarrow l$.

D. Alert and Trace

This is the core module of the RDD system. The module continuously receives the optimal aligned subgraph and its Graph Alignment Score from the preceding component. After the initialization of Step (1), whenever a new Graph Alignment Score is input, the system takes it as input and performs the computations of Step (2) through Step (3) to continuously update the posterior probability. Once the posterior probability exceeds a predefined alarm threshold, the alert mechanism is immediately triggered, and the results of each aligned subgraph are traced back accordingly. The configuration of the alarm threshold will be discussed in Step (4).

a) step1: First, some values need to be initialized before the process begins. To transform the Graph Alignment Score into the probability of being judged as an attack, we utilize historical benign activity samples (corresponding to the null hypothesis H_0) and abnormal attack samples (corresponding to the alternative hypothesis H_1). The likelihood functions $P(S | H_0)$ and $P(S | H_1)$ are modeled using *Beta* distributions to obtain their respective probability distribution models. Here, $P(S | H_0)$ denotes the probability density of obtaining a Graph Alignment Score S_t when the system is under a normal state, whereas $P(S | H_1)$ denotes the probability density of obtaining a Graph Alignment Score S_t when the system is under an attack state. These two distributions together form the likelihood basis for Bayesian-inference-based attack detection. At the initial stage of detection, considering the low

baseline probability of an attack, the prior probability is set to $P(S | H_1) = 0.01$ and $P(S | H_0) = 0.99$ as the initialization, and the initial log-likelihood ratio is defined in (4).

$$LLR_{old} = \ln \frac{P_{H_1}}{P_{H_0}} \quad (4)$$

where LLR_{old} denotes the initial log-likelihood ratio; P_{H_1} represents the initial probability of anomaly; and P_{H_0} represents the initial probability of the normal state.

b) step2: For a newly arrived Graph Alignment Score S_t , the conditional probabilities under the predefined distribution functions are first computed, the likelihood under the anomaly hypothesis $P(S_t | H_1)$ and the likelihood under the normal hypothesis $P(S_t | H_0)$. Subsequently, the log-likelihood ratio of this observed evidence is defined in (5).

$$\Delta LLR = \ln \frac{P(S_t | H_1)}{P(S_t | H_0)} \quad (5)$$

where ΔLLR denotes the log-likelihood ratio calculated based on the current Graph Alignment Score S_t ; $P(S_t | H_1)$ represents the likelihood of S_t as evidence under the anomaly hypothesis H_1 ; and $P(S_t | H_0)$ represents the likelihood of S_t as evidence under the normal hypothesis H_0 .

The above result is accumulated into the total log-likelihood ratio, which is defined in (6).

$$LLR_{new} = LLR_{old} + \Delta LLR \quad (6)$$

where LLR_{new} denotes the total log-likelihood ratio that we maintain; LLR_{old} represents the initial log-likelihood ratio; and ΔLLR denotes the log-likelihood ratio calculated based on the current Graph Alignment Score S_t .

c) *step3*: Using the updated total log-likelihood ratio LLR_{new} , the real-time posterior probability is computed through Bayesian updating, which is defined in (8).

$$P = \exp(LLR_{new}) \quad (7)$$

$$P(H_1 | S_1, S_2, \dots, S_t) = \frac{P}{1 + P} \quad (8)$$

denotes the natural exponential function:

$$\exp(x) = e^x \quad (9)$$

In Equations (7) and (8), LLR_{new} denotes the total logarithmic likelihood ratio; P represents the likelihood ratio computed from $\exp()$; and $P(H_1 | S_1, S_2, \dots, S_t)$ denotes the posterior probability, that is, the probability that the current system is in an anomalous state H_1 given the observation of a sequence of graph alignment scores $S_1, S_2, \dots, S_t$.

d) *step4*: To determine the optimal alarm threshold, recall that in the graph matching stage, in (3) L_{thr} denotes the maximum number of different entry processes that the attacker is assumed to be willing to independently exploit. Based on the computation method of the Edge Alignment Score in (3), we consider the Edge Alignment Score threshold to be the reciprocal of L_{thr} .

On the other hand, the Graph Alignment Score is computed as the average of all Edge Alignment Scores, and thus the Edge Alignment Score threshold is also used as the threshold S_{thr} for the Graph Alignment Score. Subsequently, S_{thr} is applied to the probability distribution model $P(S | H_1)$ to convert the Graph Alignment Score threshold into a posterior probability threshold for determining whether the current system is anomalous $P(S_{thr} | H_1)$.

IV. EVALUATION

We evaluate RDD on three publicly available datasets that are widely adopted by state-of-the-art host-based threat detectors. Our evaluation primarily focuses on the following aspects:

- **Q1.** Threat detection performance in comparison with two other detectors (§IV-A, IV-B, IV-C).
- **Q2.** Runtime overhead (§IV-D).
- **Q3.** Cross-environment generalizability and integration with host-based security systems (§IV-E, IV-F).

Here we present the datasets, experimental setup, and baseline implementations used for comparison. We compare RDD against two host-based threat detectors, THREATTRACE and POIROT. The experiments are conducted on three publicly available datasets: ATLAS, DARPA TC E3, and DARPA TC E5. The ATLAS dataset contains large-scale logs of real system behaviors and abnormal events, which enables the evaluation of host-based threat detectors under both normal operations and anomalous scenarios, thereby assessing their performance and robustness. DARPA TC E3 and DARPA TC E5 belong to the DARPA Transparent Computing program, providing detailed attack events and system activity logs that cover diverse attack types and intrusion paths. These datasets

serve as widely used large-scale benchmarks in cybersecurity research and facilitate the validation of detection systems under complex attack scenarios. The experimental system runs in a Windows environment, implemented in Python within the PyCharm IDE, with computation accelerated by an NVIDIA RTX4060 GPU to ensure efficient processing and model training on large-scale datasets.

In this study, we selected nine attack scenarios (M1–M6 and S1–S3) from the ATLAS dataset. From the DARPA TC E3 dataset, we extracted two ClearScope attacks, two Trace attacks, three Cadets attacks, and three Theia attacks. From the DARPA TC E5 dataset, we further collected audit logs from eight ClearScope attacks and two Trace attacks as provenance data. In addition, attack paths described in threat intelligence reports were incorporated to systematically evaluate the detection and provenance-tracing capability of our system across diverse attack scenarios.

We conducted comparative experiments between the proposed system RDD and the baselines POIROT and THREATTRACE. POIROT is a non-open-source static detection system, whose core detection logic was reimplemented based on the original paper. To ensure a fair comparison, in the experiments with POIROT, we partitioned the provenance graph of each attack scenario into multiple subgraphs corresponding to different attack stages, and conducted detection on each subgraph. This approach enables evaluation at the same level of granularity as RDD. In contrast, THREATTRACE adopts node-level anomaly detection, and its performance is assessed by whether anomalous nodes can be accurately identified in the processed provenance graphs. With this setting, we are able to comprehensively evaluate detection effectiveness at different granularities—covering both node-level anomaly detection and the tracking of complex attack paths—thus validating the advantages of the proposed method in dynamic monitoring and anomaly detection.

A. ATLAS Datasets

We conducted comparative experiments of RDD against two state-of-the-art detectors, THREATTRACE and POIROT. The experimental results are presented in Table II.

Dataset: In the ATLAS dataset (Table I), each attack is represented by a provenance graph, which is further divided into multiple subgraphs to reflect different stages of the attack over time. These subgraphs are then matched against the complete provenance graph. Since THREATTRACE is a node-level anomaly detection method, its accuracy is measured by the correct identification of anomalous nodes. POIROT is evaluated based on its static detection logic, with subgraph matching performed to ensure a fair comparison. Unlike the DARPA TC datasets, ATLAS provides rich system activity logs and anomaly events, making attack patterns more apparent and enabling a more effective evaluation of host-level detectors under realistic system operations in terms of both performance and robustness.

Results: RDD achieves the highest accuracy across all attack scenarios (92.65%–95.69%), outperforming POIROT

TABLE I: Overview of ATLAS Datasets

Scene	# of nodes	# of edges
M1	3351	16305
M2	3078	15839
M3	1921	13771
M4	2008	14235
M5	3134	15738
M6	2448	17150
S1	1210	5714
S2	3664	15389
S3	1460	8399

TABLE II: Accuracy Comparison on ATLAS Datasets

Dataset	Detector	Accuracy(%)
M1	RDD	94.17
	POIROT	78.32
	THREATRACE	89.42
M2	RDD	93.73
	POIROT	77.91
	THREATRACE	88.22
M3	RDD	94.31
	POIROT	78.84
	THREATRACE	90.64
M4	RDD	95.30
	POIROT	79.36
	THREATRACE	91.56
M5	RDD	94.67
	POIROT	79.66
	THREATRACE	89.59
M6	RDD	92.93
	POIROT	77.72
	THREATRACE	87.51
S1	RDD	95.69
	POIROT	80.37
	THREATRACE	90.33
S2	RDD	93.67
	POIROT	78.47
	THREATRACE	87.25
S3	RDD	92.65
	POIROT	77.58
	THREATRACE	87.29

(77.58%–80.37%) and THREATRACE (87.25%–91.56%). This advantage mainly stems from RDD’s dynamic detection and evidence accumulation mechanism, which continuously tracks anomalous nodes across different attack stages, reconstructing the full attack sequence and significantly improving overall detection accuracy.

B. DARPA TC E3 Datasets

We evaluated the detection performance of RDD on the DARPA TC E3 Datasets (Table III) [16], and the experimental results are presented in Table IV.

Dataset: The DARPA TC datasets were generated from red-team versus blue-team engagements under the DARPA Transparent Computing project, providing complete system operation logs including both red-team attacks and benign background activities. We selected multiple attack scenarios from the E3 task, including ClearScope, Trace, Cadets, and Theia. Each attack scenario contains a full provenance graph, which is further divided into multiple subgraphs according to different stages of the attack to simulate real-time experimental conditions, enabling precise evaluation of the detectors. Unlike

TABLE III: Overview of DARPA TC E3 Datasets

Scene	# of nodes	# of edges
ClearScope1	15612	36213
ClearScope2	12463	46913
Trace1	42314	78089
Trace2	30348	60559
Cadets1	17862	34093
Cadets2	29620	39304
Cadets3	18497	27313
Theia1	30348	50559
Theia2	33728	41665
Theia3	26488	50709

TABLE IV: Accuracy Comparison on DARPA TC E3 Datasets

Dataset	Detector	Accuracy(%)
ClearScope1	RDD	93.22
	POIROT	72.41
	THREATRACE	86.63
ClearScope2	RDD	95.11
	POIROT	73.25
	THREATRACE	87.34
Trace1	RDD	92.87
	POIROT	72.37
	THREATRACE	85.33
Trace2	RDD	93.65
	POIROT	68.84
	THREATRACE	86.63
Cadets1	RDD	89.37
	POIROT	73.58
	THREATRACE	87.73
Cadets2	RDD	92.95
	POIROT	73.22
	THREATRACE	89.61
Cadets3	RDD	93.07
	POIROT	72.27
	THREATRACE	86.63
Theia1	RDD	92.37
	POIROT	69.86
	THREATRACE	87.57
Theia2	RDD	90.57
	POIROT	72.51
	THREATRACE	88.18
Theia3	RDD	90.38
	POIROT	72.65
	THREATRACE	87.09

the ATLAS dataset, attacks in the DARPA TC dataset are long-running, with anomalous activities interleaved with a large volume of normal operations, and anomalous nodes account for only a small fraction of the overall graph. ATLAS, by contrast, provides rich system activity logs and anomaly records, making attack patterns more apparent and temporally continuous. The sparsity and complex background in DARPA TC make it a challenging dataset for testing the robustness and tracing ability of detectors in long-term, low signal-to-noise ratio scenarios.

Results: Across all attack scenarios, RDD achieves an accuracy of 89.37%–95.11% on the DARPA TC E3 dataset, significantly outperforming POIROT (68.84%–73.58%) and THREATRACE (85.33%–87.73%). Compared to the overall accuracy of 92.65%–95.69% on the ATLAS dataset, the accuracy on E3 is slightly lower but still maintains an advantage. This demonstrates that RDD can capture early anomalous nodes and reconstruct attack paths through dynamic detection

TABLE V: Overview of DARPA TC E5 Datasets

Scene	# of nodes	# of edges
ClearScope1	31537	50115
ClearScope2	35901	51105
ClearScope3	34053	49875
ClearScope4	41985	59205
ClearScope5	43754	61446
ClearScope6	32899	57651
ClearScope7	40712	68756
ClearScope8	41317	63019
Trace1	56399	82580
Trace2	40247	93051

TABLE VI: Accuracy Comparison on DARPA TC E5 Datasets

Dataset	Detector	Accuracy(%)
ClearScope1	RDD	88.51
	POIROT	63.90
	THREATTRACE	84.20
ClearScope2	RDD	91.12
	POIROT	66.53
	THREATTRACE	88.17
ClearScope3	RDD	85.25
	POIROT	61.63
	THREATTRACE	79.81
ClearScope4	RDD	87.88
	POIROT	64.23
	THREATTRACE	84.63
ClearScope5	RDD	87.97
	POIROT	59.30
	THREATTRACE	85.21
ClearScope6	RDD	84.53
	POIROT	59.17
	THREATTRACE	81.16
ClearScope7	RDD	85.84
	POIROT	56.31
	THREATTRACE	84.33
ClearScope8	RDD	86.71
	POIROT	58.20
	THREATTRACE	78.73
Trace1	RDD	81.44
	POIROT	54.12
	THREATTRACE	76.38
Trace2	RDD	85.15
	POIROT	56.79
	THREATTRACE	79.24

and evidence accumulation, even when all anomalous nodes in the complete graph do not appear simultaneously. Overall, RDD shows high robustness and traceability for long-running system threat detection and provenance tasks and can adapt to the challenges posed by different dataset characteristics.

C. DARPA TC E5 Datasets

The Table VI presents a comparison of detection accuracy on the DARPA TC E5 Datasets (Table V). Overall, RDD achieves an accuracy of 81.44%–91.12% across the ClearScope and Trace scenarios, significantly outperforming POIROT (54.12%–66.53%) and generally exceeding THREATTRACE (76.38%–88.17%). In certain ClearScope scenarios (e.g., ClearScope2), RDD reaches or exceeds 91%, whereas in some Trace scenarios (e.g., Trace1) the accuracy is relatively lower ($\approx 81\%$), indicating that the differences among data scenarios have a noticeable impact on detection performance.

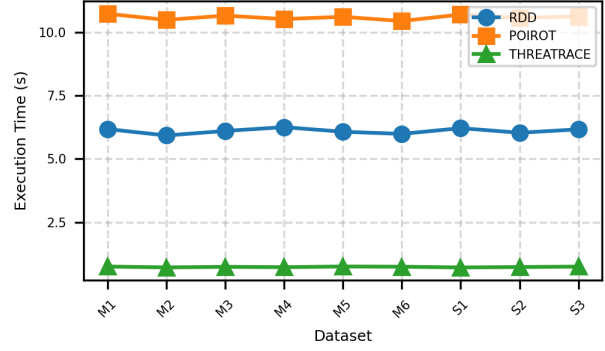


Fig. 2: Execution Time Comparison on ATLAS Dataset.

Compared with ATLAS and E3, E5 poses greater challenges, mainly in the following aspects:

- **Scale and sparsity:** The provenance graphs in E5 are larger and cover longer time spans, with anomalous nodes being extremely sparse in the overall graph (low signal-to-noise ratio), which increases the difficulty of detecting anomalies amid a vast amount of normal background activity.
- **Attack complexity:** E5 includes more complete APT/multi-stage attack chains, with malicious actions distributed across longer time periods and multiple hosts. At any single time point or along a single path, obvious anomalous signals may not be present.

Based on the aforementioned characteristics, POIROT exhibits relatively low performance on E5, as its static, whole-graph matching strategy struggles to effectively adapt to ultra-large-scale, temporally distributed attack flows. THREATTRACE, as a node-level anomaly detection method, can identify local anomalies to some extent and thus achieves moderate performance. However, its dependence on training-phase resources and lack of cross-time evidence accumulation limit its effectiveness against long-term, dispersed attacks. In contrast, RDD leverages a dynamic monitoring and evidence accumulation design: it can aggregate local anomaly evidence across different time points to reconstruct attack paths, while employing a scalable graph processing strategy to maintain high accuracy even under the high complexity of E5 scenarios.

Comparison with ATLAS and E3: On ATLAS (medium scale, with more evident attack patterns), RDD achieves a higher accuracy range (approximately 92%–96%), while on E3 (moderate-to-large scale, with attack duration and background noise between ATLAS and E5), RDD also demonstrates stable performance. In comparison, E5 generally yields lower accuracy than ATLAS, and some scenarios fall below E3, highlighting the negative impact of dataset scale, attack duration, and background noise on detector performance. This comparison further indicates that in large-scale, long-running real-world environments, purely static or single-time-point approaches are prone to failure, whereas dynamic methods that

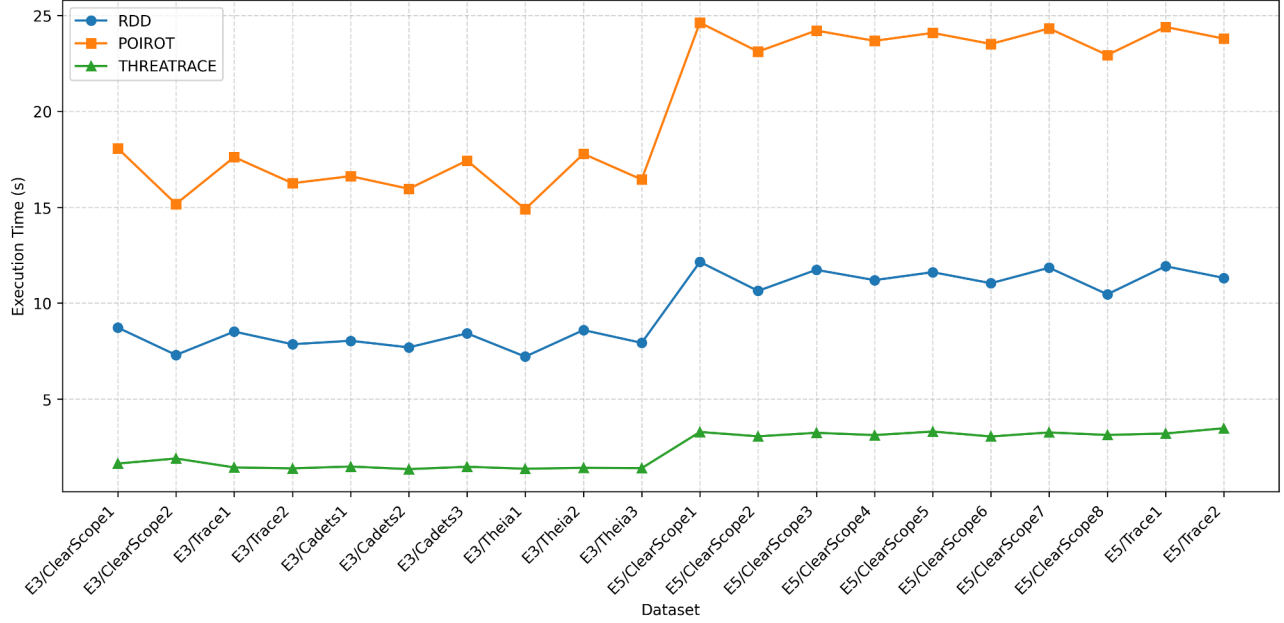


Fig. 3: Execution Time Comparison on DARPA TC E3 and E5 Datasets.

integrate temporal and cross-phase evidence, such as RDD, are necessary to improve robustness.

The E5 experiments demonstrate that RDD possesses strong tracing and detection capabilities in the presence of large-scale, low signal-to-noise, and temporally distributed attacks. Nonetheless, the performance drop in some Trace scenarios suggests potential avenues for further optimization—for example, enhancing sensitivity to extremely sparse anomalies, improving cross-host evidence aggregation, and optimizing computation on ultra-large graphs—thereby enabling more comprehensive performance gains in industrial-scale deployments.

D. Runtime Overhead

We evaluated the execution time of RDD, POIROT, and THREATTRACE on the ATLAS, DARPA TC E3, and E5 datasets, as shown in Fig. 2 and Fig. 3. Overall, RDD achieved runtimes of 5.92–6.25s on ATLAS and 7.21–12.15s on the DARPA TC datasets, demonstrating good scalability as graph size and complexity increase. In contrast, POIROT requires multiple traversals of the provenance graph due to its whole-graph matching strategy, resulting in longer runtimes (10.44–10.73s on ATLAS and 14.90–24.62s on DARPA TC). THREATTRACE, on the other hand, achieved the shortest runtimes across all datasets (0.71–3.47s).

However, the lower runtime of THREATTRACE does not necessarily indicate lower resource consumption. As a deep learning-based detector, it incurs substantial hidden overhead during model training and requires significant CPU and memory resources during inference, which are not reflected in the reported runtimes. Although RDD is slightly slower, its

scalable evidence-accumulation design enables efficient processing of large-scale and temporally distributed provenance graphs with moderate resource usage. Therefore, for large-scale and long-duration attack scenarios, such as DARPA TC E5, RDD achieves a favorable balance between detection accuracy and runtime performance.

E. Cross-Environment Generalizability

Although the experiments reported above focus on specific datasets and operating systems, RDD’s design inherently supports adaptability across diverse system environments. RDD employs a provenance-graph-based matching algorithm to dynamically identify anomalous behaviors, combined with streaming evidence accumulation and incremental Bayesian updates. These mechanisms are largely independent of the underlying operating system, as the system monitors process-level and file-level events in a generic manner and relies on an event abstraction layer to handle differences in system calls, file paths, and process identifiers.

Furthermore, RDD incorporates a node alignment scoring mechanism to efficiently find the optimal aligned subgraph without exhaustive graph traversal, which contributes to its scalability and performance consistency under varying workloads and system configurations. Its modular architecture and parameterized alert thresholds further enable deployment across heterogeneous systems while maintaining detection effectiveness. Therefore, even in the absence of explicit cross-platform experiments, RDD is expected to perform robustly across different operating systems and workload intensities. Future work will empirically validate its adaptability in heterogeneous and industrial-scale environments.

F. Integration with Host-Based Security Systems

RDD's streaming provenance analysis and modular architecture enable efficient processing of real-time event data and generation of anomaly detection outputs that can be integrated with centralized logging and alerting platforms, such as Endpoint Detection and Response (EDR) or Security Information and Event Management (SIEM) systems. Its parameterized alert thresholds and lightweight evidence accumulation mechanisms allow deployment in large-scale environments without incurring excessive computational overhead, making integration with existing host-based security infrastructures highly feasible. Although industrial-scale deployment experiments are beyond the scope of this work, RDD's design indicates potential applicability in enterprise networks for continuous monitoring and early detection of advanced persistent threats. Future work will evaluate end-to-end integration with industrial EDR/SEIM platforms and assess performance under high-volume operational workloads.

V. CONCLUSION

We present RDD, a real-time host monitoring system for detecting and tracing stealthy Advanced Persistent Threats (APTs). RDD uses a provenance-graph-based matching algorithm to dynamically identify anomalous behaviors. Graph alignment scores from different time periods are converted into log-likelihood ratios, serving as quantifiable evidence that is incrementally accumulated. A Bayesian update then revises the posterior probability of a host being abnormal, framing anomaly detection as a probabilistic estimation. The system also uses a node alignment scoring mechanism to efficiently find the optimal aligned subgraph without exhaustive graph traversal. Experimental results show that RDD achieves accurate and efficient anomaly detection while maintaining real-time performance.

ACKNOWLEDGMENT

This work was supported in part by the Central Government Guiding Local Scientific and Technological Development Fund of Shanxi Province under Grant YDZJSX2025D029, and in part by the National Key R&D Program of China under Grant 2023YFB2703800.

REFERENCES

- [1] M. Barre, A. Gehani, and V. Yegneswaran, "Mining data provenance to detect advanced persistent threats," in Proc. 11th Int. Workshop Theory Pract. Provenance (TaPP), 2019, pp. 1–11.
- [2] Q. Wang, Y. Chen, Z. Lin, X. Li, and Y. Wang, "You are what you do: Hunting stealthy malware via data provenance analysis," in Proc. 27th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS), San Diego, CA, USA, Feb. 2020, pp. 1–17.
- [3] W. U. Hassan, A. Bates, and D. Marino, "Tactical provenance analysis for endpoint detection and response systems," in Proc. IEEE Symp. Secur. Privacy (SP), May 2020, pp. 1172–1189.
- [4] G. Xin, G. Xu, Y. Zhang, C. Wen, C. Zhang, X. Xie, N. N. Xiong, S. Liu, and P. Gao, "IRHunter: Universal detection of instruction reordering vulnerabilities for enhanced concurrency in distributed and parallel systems," IEEE Trans. Parallel Distrib. Syst., vol. 36, no. 6, pp. 1220–1236, Jun. 2025, doi: 10.1109/TPDS.2025.3556861.
- [5] R. Wei, L. Cai, L. Zhao, A. Yu, and D. Meng, "Deephunter: A graph neural network based approach for robust cyber threat hunting," in Proc. Int. Conf. Secur. Privacy Commun. Syst. (SecureComm), Cham: Springer, 2021, pp. 3–24.
- [6] W. Qiao, Y. Feng, T. Li, Z. Ma, Y. Shen, J. Ma, and Y. Liu, "Slot: Provenance-Driven APT detection through graph reinforcement learning," arXiv:2410.17910, Oct. 2024. [Online]. Available: <https://arxiv.org/abs/2410.17910>
- [7] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. N. Venkatakrishnan, "Poirot: Aligning attack behavior with kernel audit records for cyber threat hunting," in Proc. USENIX Secur. Symp., 2019, pp. 1387–1404.
- [8] G. Xu, G. Xin, L. Jiao, J. Liu, S. Liu, M. Feng, and X. Zheng, "OFEL: A semi-black-box Android adversarial sample attack framework against DLaaS," IEEE Trans. Comput., vol. 73, no. 4, pp. 956–969, Apr. 2024. DOI:10.1109/TC.2023.3236872.
- [9] G. Xu, L. Jiao, J. Liu, S. Liu, and X. Zheng, "DomFuzz: Dominator-based directed grey-box fuzzing for ICPSSs," IEEE Network, vol. 39, no. 4, pp. 54–61, Jul. 2025, doi: 10.1109/MNET.2025.3540657.
- [10] E. Altinisik, F. Deniz, and H. T. Sencar, "ProvG-Searcher: A graph representation learning approach for efficient provenance graph search," in Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), 2023, pp. 2247–2261. doi: 10.1145/3576915.3623187
- [11] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "Unicorn: Runtime provenance-based detector for advanced persistent threats," in Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS), 2020, pp. 1–19.
- [12] E. Manzoor, S. M. Milajerdi, and L. Akoglu, "Fast memory-efficient anomaly detection in streaming heterogeneous graphs," in Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min. (KDD), Aug. 2016, pp. 1035–1044.
- [13] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Venkatakrishnan, "Holmes: Real-time APT detection through correlation of suspicious information flows," in Proc. IEEE Symp. Secur. Privacy (SP), May 2019, pp. 1137–1152.
- [14] S. Wang, T. Zhang, J. Zhao, L. Zhang, and X. Chen, "ThreTRACE: Detecting and tracing host-based threats in node level through provenance graph learning," IEEE Trans. Inf. Forensics Secur., early access, Piscataway, NJ, USA: IEEE, DOI:10.1109/TIFS.2022.3208815.
- [15] T. Pasquier, B. Shand, J. Singh, and M. Seltzer, "Practical whole-system provenance capture," in Proc. ACM Symp. Cloud Comput. (SoCC), 2017, pp. 405–418.
- [16] A. D. Keromytis. (2018). Transparent Computing Engagement 3 Data Release. [Online]. Available: <https://github.com/darpa-i2o/Transparent-Computing/blob/master/README-E3.md>